



**Université Ibn Tofaïl
Faculté des Sciences
Kénitra**

A.U. 2025/2026

ALGORITHMIQUE ET STRUCTURES DE DONNEES

MIP-Mathematiques - S4

Pr. N. ENNEYA



**Université Ibn Tofaïl
Faculté des Sciences
Kénitra**

A.U. 2025/2026

Introduction générale

MIP-Mathematiques - S4

Pr. N. ENNEYA

I. Définitions:

- Un algorithme est la composition d'un ensemble fini d'étapes, chaque étape étant formée d'un nombre fini d'opérations.
- Une structure de données est une organisation des données dans la mémoire afin d'être utilisées par des programmes de manière efficace.
- Programme = Algorithme + Structures de données

Remarque:

- Structures de données $\neq$ Bases de données
(RAM) (Disque dur,...)

II. Plan du cours:

Nous allons étudier dans ce cours les grandes classes de structures de données utilisées dans la programmation:

- Rappels: Pointeurs , enregistrements et fonctions récursives

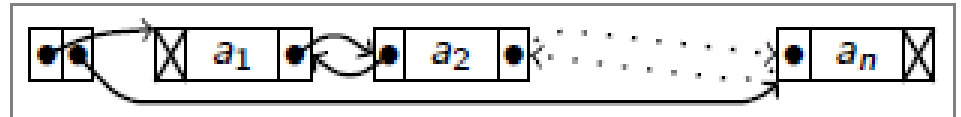
- Les structures de données linéaires :

- Les tableaux ;

- Les listes chaînées: `lst` 

- Listes simplement chaînées

- Listes doublement chaînées

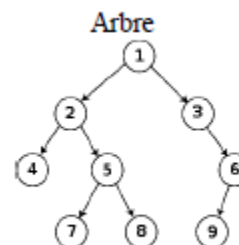


- Plies et Files ;

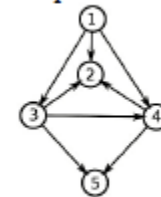
- Les structures de données non linéaires:

- Les structures arborescentes

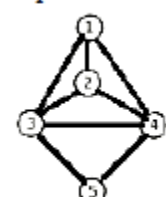
- Les graphes.



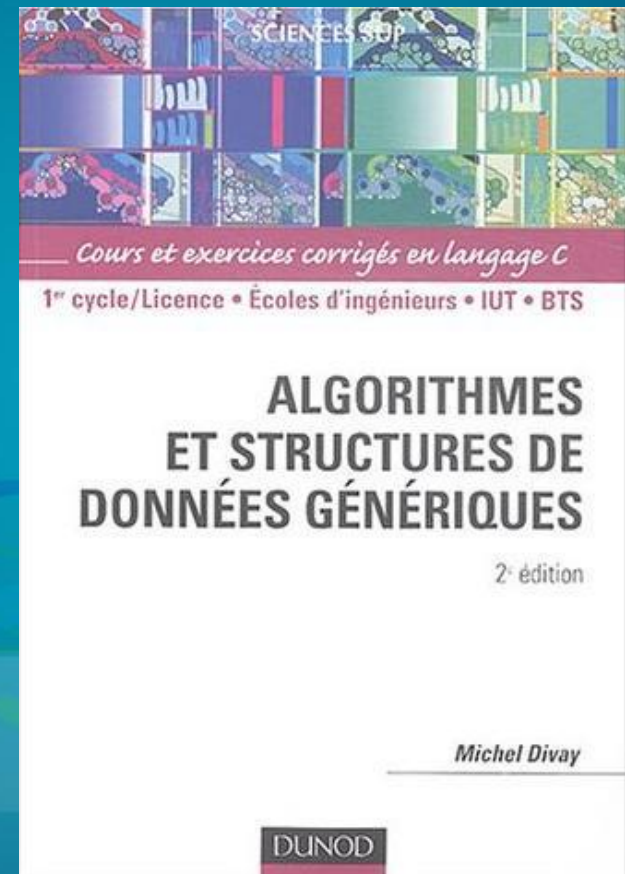
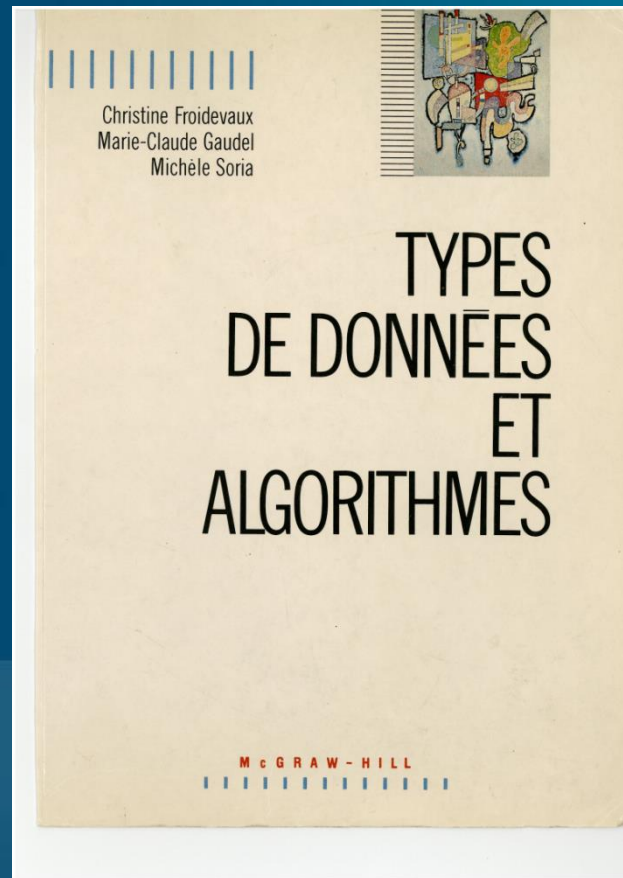
Graphe orienté



Graphe non orienté



III. Références:





Université Ibn Tofaïl
Faculté des Sciences
Kénitra

A.U. 2025/2026

Chapitre 1: Rappels

MIP-Mathematiques - S4

Pr. N. ENNEYA

1. Introduction

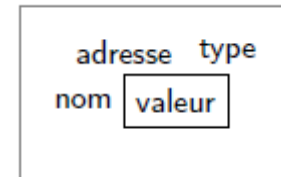
Dans ce chapitre nous étudions les principales notions utilisées dans les structures de données à savoir:

- Pointeurs;
- Allocation dynamique de mémoire;
- Enregistrements : struct;
- Enregistrements et pointeurs;
- Fonctions récursives.

2. Pointeurs et variable dynamique

- Une variable statique (usuelle) est caractérisée par quatre propriétés : (nom, adresse, type, valeur)

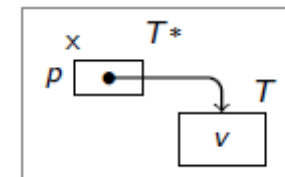
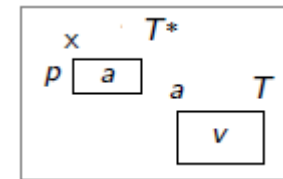
- `int v = 5;`



- Un pointeur est une variable spéciale qui peut contenir l'adresse d'une autre variable ou d'une zone mémoire de manière générale.

- Si un pointeur **p** contient l'adresse **a** de la variable **v**, on dit que :

p pointe sur v.

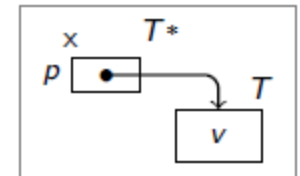
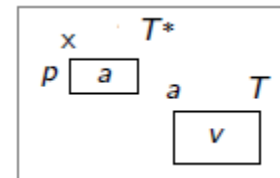


- `int *p;`

- `p=&v;`

2. Pointeurs et variable dynamique

- Un pointeur **p** qui pointe vers une variable **v** d'adresse **a** et de type **T**
 - est de type **T ***;
 - a pour valeur l'adresse **a** de la variable **v**.
- Déclaration:
 - **T* p;**
- Initialisations:
 - **p=&v;** // avec **v** de type **T**
 - **p=NULL;**



NULL est une valeur constante commune à tous les pointeurs, un pointeur qui a le valeur **NULL** ne pointe vers aucune variable.

2. Pointeurs et variable dynamique

- Une variable dynamique (VD) est anonyme :
(adresse, type, valeur). On y accède grâce à un pointeur.

- **Création d'une variable dynamique:**

$T^* p = (T^*) \text{ malloc } (\text{sizeof } (T));$

crée une VD de type T ; p pointe sur cette variable.

p contient l'adresse de la zone créée ;

$T^* q = (T^*) \text{ malloc } (\text{sizeof } (T));$

La fonction [malloc\(\)](#) de la bibliothèque [stdlib.h](#) retourne NULL si l'allocation est échouée.

- **Affectation de pointeurs**

$p = \text{NULL} ;$ NULL indique un pointeur nul;

$p = q ;$ p et q pointent sur la même zone mémoire.

- **Affectation de zones pointées**

$*p = *q ;$

mettre à l'adresse pointée par p, ce qu'il y a à l'adresse pointée par q.

2. Pointeurs et variable dynamique

- **Comparaison de pointeurs (== et !=)**

if (**p == q**) { ... si p et q sur le même variable...

if (**p != q**) { ... si p et q ne repèrent pas le même objet ...

On peut tester si un pointeur est supérieur ou inférieur à un autre pointeur notamment lorsqu'on parcourt un tableau d'éléments de type T.

- **Addition et soustraction de pointeurs: p++; p—**

Si **p** pointe de type **T**, **p+1** pointe sur l'objet suivant de type T, de même que **p++;**

Si p et q sont 2 pointeurs de type T, **p-q** indique le nombre d'objets de type T entre p et q.

- **Suppression de la zone allouée dynamiquement**

free (p) ;

rend au système d'exploitation l'espace mémoire occupé par l'objet pointé par p, et alloué par **malloc()**,.

2. Pointeurs et variable dynamique

● Double allocation, fuite de mémoire:

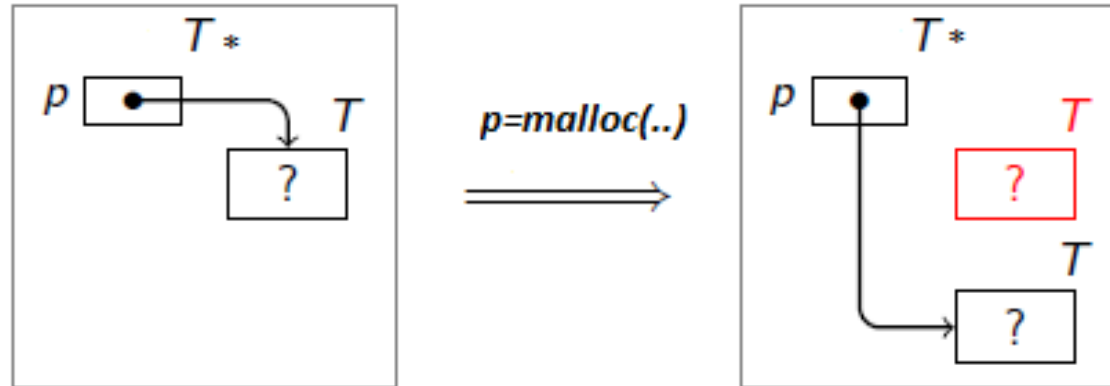
Attention à ne pas perdre l'accès aux VDs :

par exemple, si on exécute deux allocations successives à p , on ne peut plus accéder à la première VD allouée :

$T^* p = (T^*) \text{ malloc } (\text{sizeof } (T));$

...

$T^* p = (T^*) \text{ malloc } (\text{sizeof } (T));$



La zone mémoire de la VD en rouge est perdue !

3. Pointeurs et tableaux

● On suppose déclaré un type **T**:

● **Définition:**

Un tableau **Tab** est une structure de donnée qui permet de stocker un certain nombre d'éléments **Tab[i]** repérés par un indice *i*. Les tableaux vérifient généralement les propriétés suivantes :

- Tous les éléments ont le même type de base ;
- Le nombre d'éléments stockés est fixé ;
- l'accès et la modification de l'élément numéro *i* est en temps constant $\Theta(1)$, indépendant de *i* et du nombre d'éléments dans le tableau

● **Déclaration statique :**

T tab[TAILLE];

Avec TAILLE est une constante fixée

3. Pointeurs et tableaux

● Problème de la taille maximum:

Si on veut insérer un élément dans un tableau où la taille est maximale, on n'aura plus de place disponible.

Comportements possibles :

- Erreur (arrêt du programme, exception)
- Réallocation du tableau avec recopie, coût : $\Theta(\text{TAILLE})$

● Déclaration dynamique :

`T* p;`

...

`p = (T*) malloc(n*sizeof(T));`

Avec **n une variable** entière donnée.

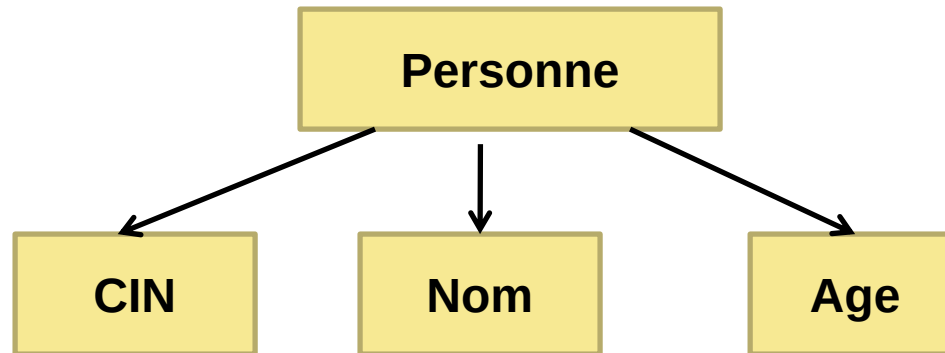
L'adresse de `P[i]` est `p + i`.

La valeur de `P[i]` est `*(p+i)`.

4. Enregistrements

● Introduction

Dans la programmation on ait besoin souvent d'un mécanisme permettant de grouper un certain nombre de variables de types différents au sein d'une même entité. On travaille par exemple sur un fichier de personnes et on voudrait regrouper deux variables de type chaîne de caractères pour le CIN et le nom avec une variable de type entier pour l'âge, etc.



- **Problème :** Si on stocke les données des personnes dans des tableaux séparés, on aura un tableaux des CIN, tableaux des noms et un tableaux des âges. Dans ce cas on se retrouve avec des informations du même personne éparpillées sur plusieurs tableaux, on ne peut pas donc parler de **l'entité personne**.

4. Enregistrements

La réponse à ce besoin est le concept d'enregistrement (en anglais record)

- **Définition:**

Un **enregistrement** ou une **structure** est un ensemble d'éléments qui peuvent être de types différents repérés par un nom.

Les éléments d'un enregistrement sont appelés des **champs**.

- **Déclaration de type structuré :**

```
struct nom_de_struct {  
    type_du_champ_1  nom_du_champ_1 ;  
    type_du_champ_2  nom_du_champ_2 ;  
    type_du_champ_3  nom_du_champ_3 ;  
    ...  
};
```

- **Déclaration d'un enregistrement :**

```
struct  nom_de_struct  v;
```

- **L'accès aux champs :**

```
v. nom_du_champ_1
```


4. Enregistrements

● **Exemple:**

```
struct Personne {  
    char CIN[8];  
    char nom[10];  
    int age;  
};  
  
int main() {  
    struct Personne P1;  
    strcpy (P1.nom, "Hamid");  
    strcpy (P1.CIN, "A123");  
    P1.age = 21;  
  
    printf ("%s %s %d\n", P1.CIN, P1.nom, P1.age);  
    return 0;  
}
```

4. Enregistrements

- Initialisation d'un enregistrement :

```
struct  Personne P1={"A12345", " Hamid" , 21};
```

- Opération sur les enregistrements:

La seule opération qui existe entre les enregistrements est l'affectation =.

```
struct  Personne P2;
```

```
P2 = P1;
```

- Définition de type:

```
typedef struct  nom_de_struct  nom_type;
```

```
nom_type  v;
```

- Définition et déclaration de type structuré :

```
typedef struct nom_de_struct {  
    type_du_champ_1  nom_du_champ_1 ;  
    type_du_champ_2  nom_du_champ_2 ;  
    type_du_champ_3  nom_du_champ_3 ;  
    ...  
} nom_type;
```

5. Enregistrements et pointeurs

- On considère la structure suivante:

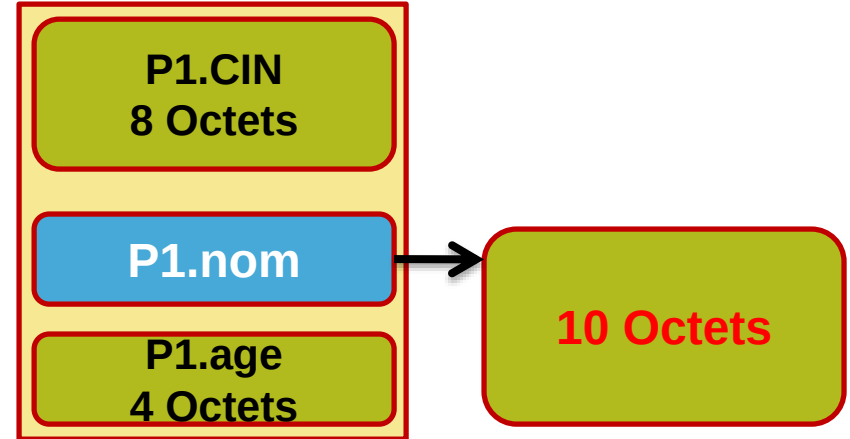
```
typedef struct Personne {  
    char CIN[8];  
    char nom[10];  
    int age;  
} Personne;  
Personne *P1 , P2={"B123" , "Farid", 26};  
//Allocation dynamique de *P1:  
P1=(Personne*)malloc(sizeof(Personne));  
strcpy (P1->nom, "Hamid");  
strcpy (P1->CIN, "A123");  
P1->age = 21;  
printf ("%s %s %d\n", P1->CIN, P1->nom, P1->age);
```

- L'accès aux champs d'un pointeur d'enregistrement se fait via ->
P1->age est équivalente à **(*P1).age**

5. Enregistrements et pointeurs

- On considère la structure suivante où le champ nom est un pointeur:

```
typedef struct Personne {  
    char CIN[8];  
    char *nom;  
    int age;  
} Personne;
```



```
Personne P1;  
strcpy (P1.CIN, "A123");  
//Allocation dynamique de P1.nom:  
P1.nom=(char *)malloc(10*sizeof(char));  
P1.nom ="Hamid";  
  
P1.age = 21;  
printf ("%s %s %d\n", P1.CIN, P1.nom, P1.age);
```

6. Structure de structure:

- On considère la structure suivante où on a ajouté à la structure `Personne` le champ `date de naissance`:

```
typedef struct date{  
    int jour, mois, annee;  
}date;
```

```
typedef struct Personne {  
    char CIN[8];  
    char nom[10];  
    int age;  
    date dn;  
}Personne;
```

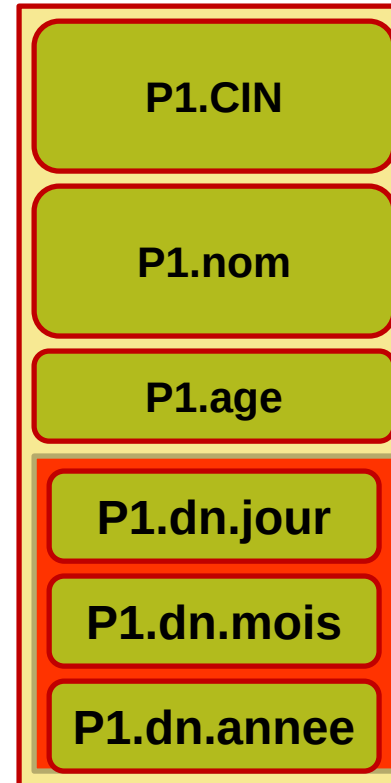
```
Personne P1;
```

```
strcpy (P1.CIN, "B123");
```

```
strcpy(P1.nom , "Hamid");
```

```
P1.age = 21;
```

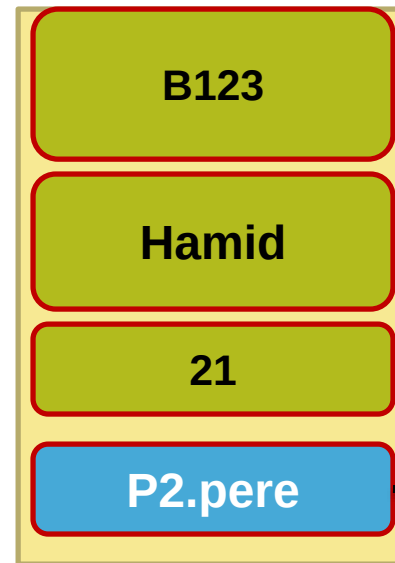
```
P1.dn.jour=15; P1.dn.mois=11; P1.dn.annee=1998;
```



6. Structure de structure:

- Afin de relier entre elles les différentes structures qui correspondent aux différentes personnes, on décrit un pointeur sur un enregistrement de type *Personne*. Ce pointeur fournit l'adresse de la personne pointée.

```
typedef struct Personne {  
    char CIN[8];  
    char nom[10];  
    int age;  
    struct Personne *pere;  
} Personne;
```



```
Personne P2, P1={"A123" , "Karim", 56, NULL};  
strcpy (P2.CIN, "B123");  
strcpy (P2.nom , "Hamid");  
P2.age = 21;  
P2.pere=&P1; //initialisation par &P1
```



6. Structure de structure:

Remarque:

Il ne faut jamais décrire un champ de type structure dans un enregistrement qui a le même type de structure.



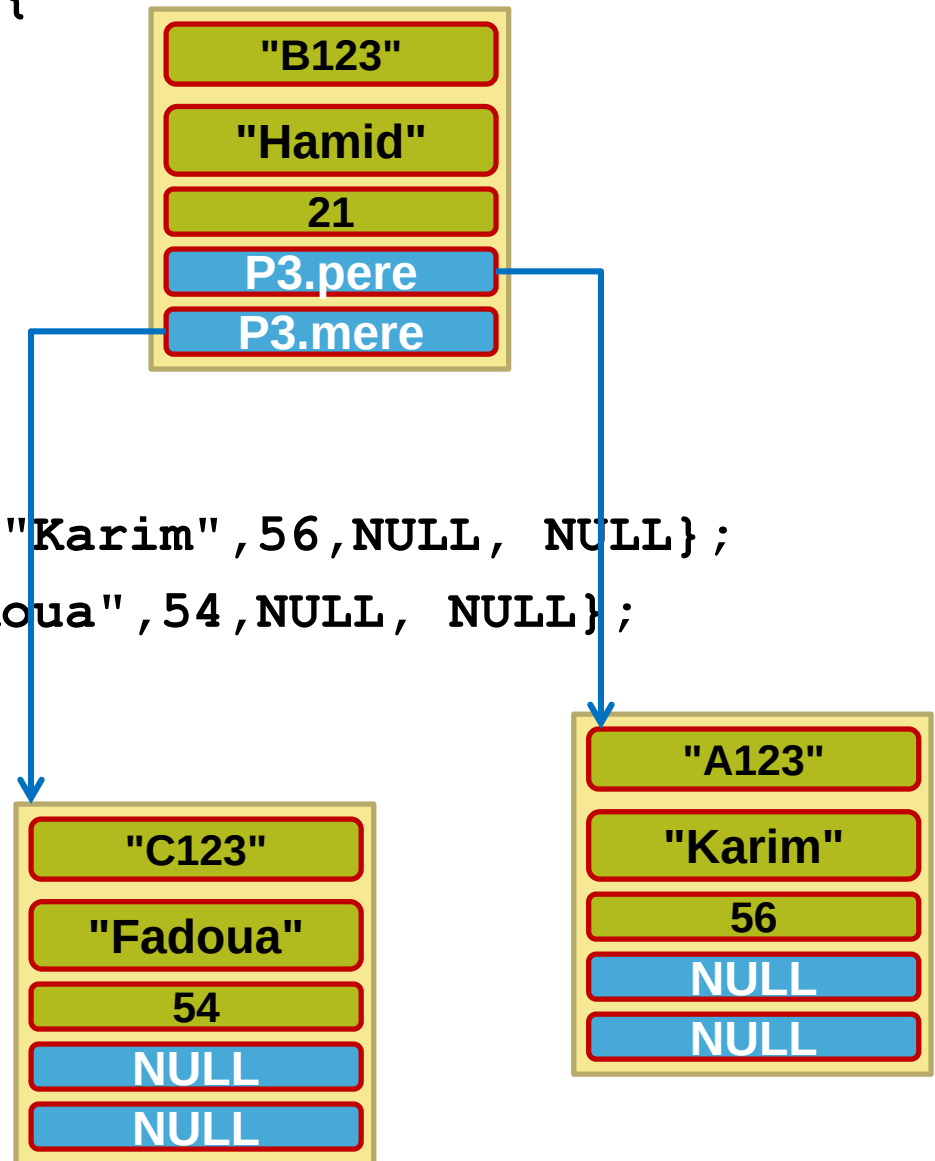
```
typedef struct Personne {  
    char CIN[8];  
    char nom[10];  
    int age;  
    struct Personne pere; //ERREUR  
} Personne;
```

Exemple:

Le code suivant permet la construction de structures de données complexes correspondant à un arbre généalogique. Les champs **pere** et **mere** sont des pointeurs de **Personne** sur la personne père et mère respectivement. Ainsi, pour chaque **Personne**, on a l'adresse en mémoire de son père et de sa mère. L'absence de pointeur (père ou mère inconnus sur l'exemple) se note **NULL**.

6. Structure de structure:

```
typedef struct Personne {  
    char CIN[8];  
    char nom[10];  
    int age;  
    struct Personne *pere;  
    struct Personne *mere;  
} Personne;  
  
Personne P3, P1={"A123","Karim",56,NULL, NULL};  
Personne P2={"C123","Fadoua",54,NULL, NULL};  
strcpy (P3.CIN, "B123");  
strcpy (P3.nom , "Hamid");  
P3.age = 21;  
P3.pere=&P1;  
P3.mere=&P2;
```



7. Tableau d'enregistrements:

- On déclare les tableaux de structure de la même manière que les types simples
int , float , ... etc.

```
typedef struct Personne {  
    char CIN[8];  
    char nom[10];  
    int age;  
} Personne;
```

```
Personne P[TAILLE];
```

- L'accès aux champs de l'élément du tableau P d'indice i se fait par :

```
P[i].CIN
```

```
P[i].nom
```

```
P[i].age
```

0

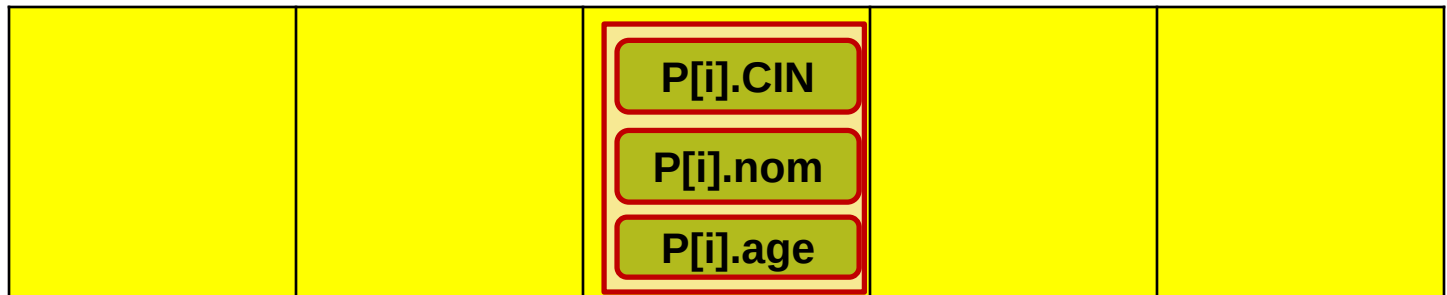
...

i

...

TAILLE-1

P =



8. Fonctions récursives

- La récursivité est une méthode de description d'algorithmes qui permet à une fonction de s'appeler elle-même un certain nombre de fois fini.
 - Une fonction récursive s'appelle elle-même : on recommence l'exécution avec de nouvelles données.
 - il y a un test de fin : la condition d'arrêt. Il est souvent préférable d'indiquer le test de fin des appels récursifs en début de la fonction.

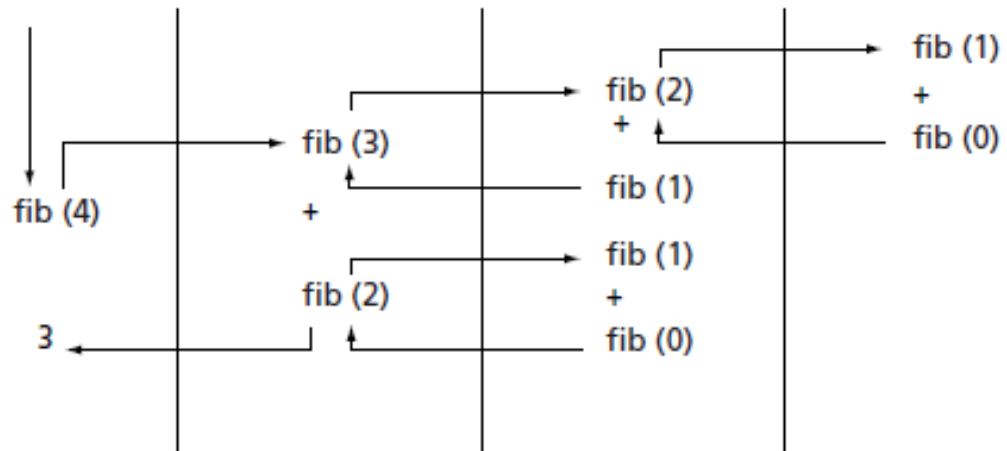
- Exemple 1: $n! = n * (n-1)!$ et $0! = 1$

```
long factorielle (int n) {  
    if (n == 0) {  
        return 1;  
    } else {  
        return n*factorielle (n-1);  
    }  
}
```

8. Fonctions récursives

● Exemple 2 :
$$\begin{cases} F_0 = 0, F_1 = 1; \\ F_n = F_{n-1} + F_{n-2} \end{cases}$$

```
long fib(int n) {  
    if (n <= 1) {  
        return n;  
    } else {  
        return fib(n-1) + fib(n-2);  
    }  
}
```



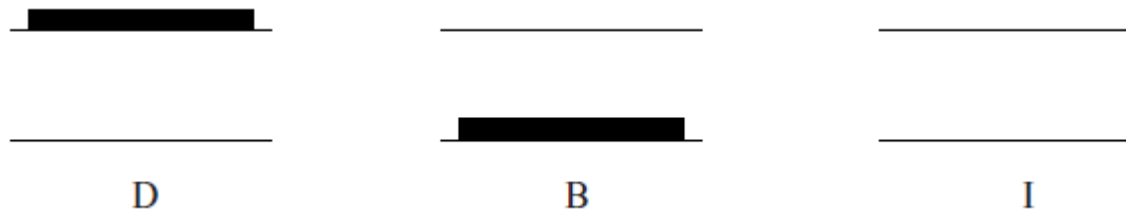
8. Fonctions récursives

● Exemple 3 : Tours de Hanoi

Les « Tours de Hanoi » est un jeu où il s'agit de déplacer un par un des disques superposés de diamètre décroissant d'un socle de départ **D** sur un socle de but **B**, en utilisant éventuellement un socle intermédiaire **I**. Un disque ne peut se trouver au dessus d'un disque plus petit que lui.

- **Le cas d'un disque:** Pour déplacer un disque de **D** vers **B**, le socle **I** (intermédiaire) est inutile.

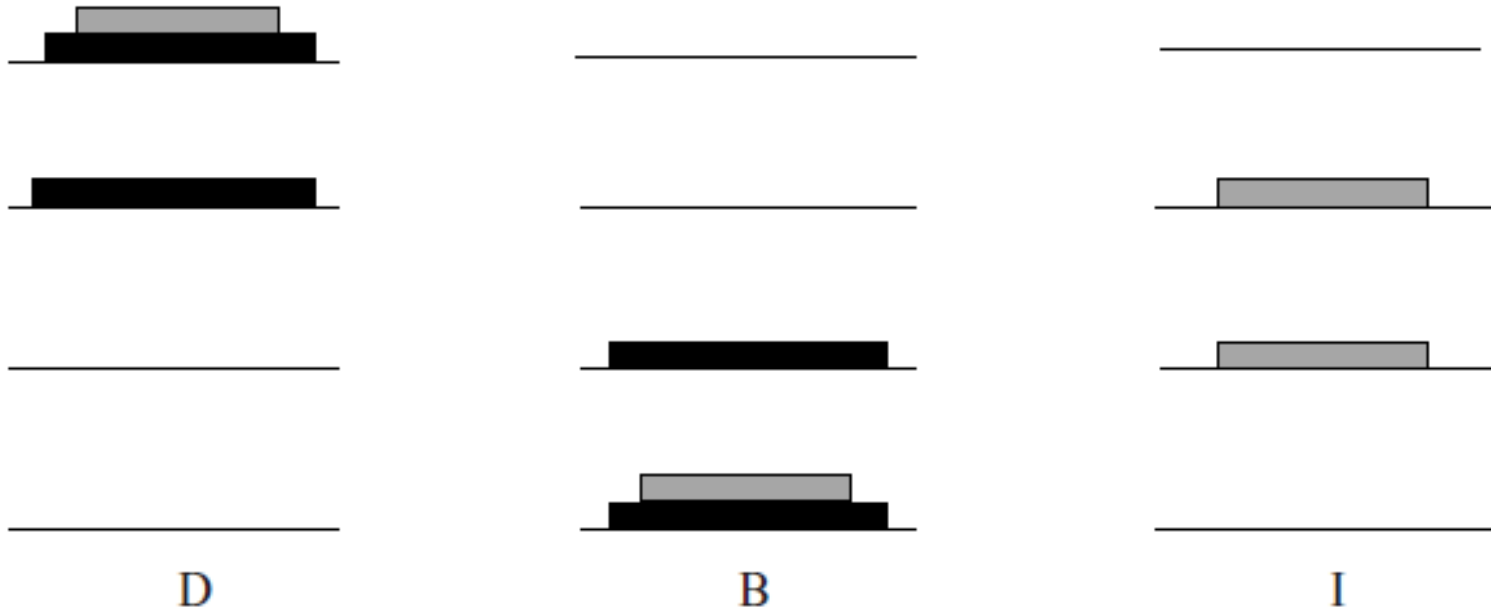
deplacer(1, D, B, I);



8. Fonctions récursives

- Le cas de 2 disques: Pour déplacer 2 disques, il faut transférer celui qui est au sommet sur le socle **I**, déplacer le disque reposant sur le socle **D** vers **B**, et ramener le disque du socle **I** au sommet de **B**.

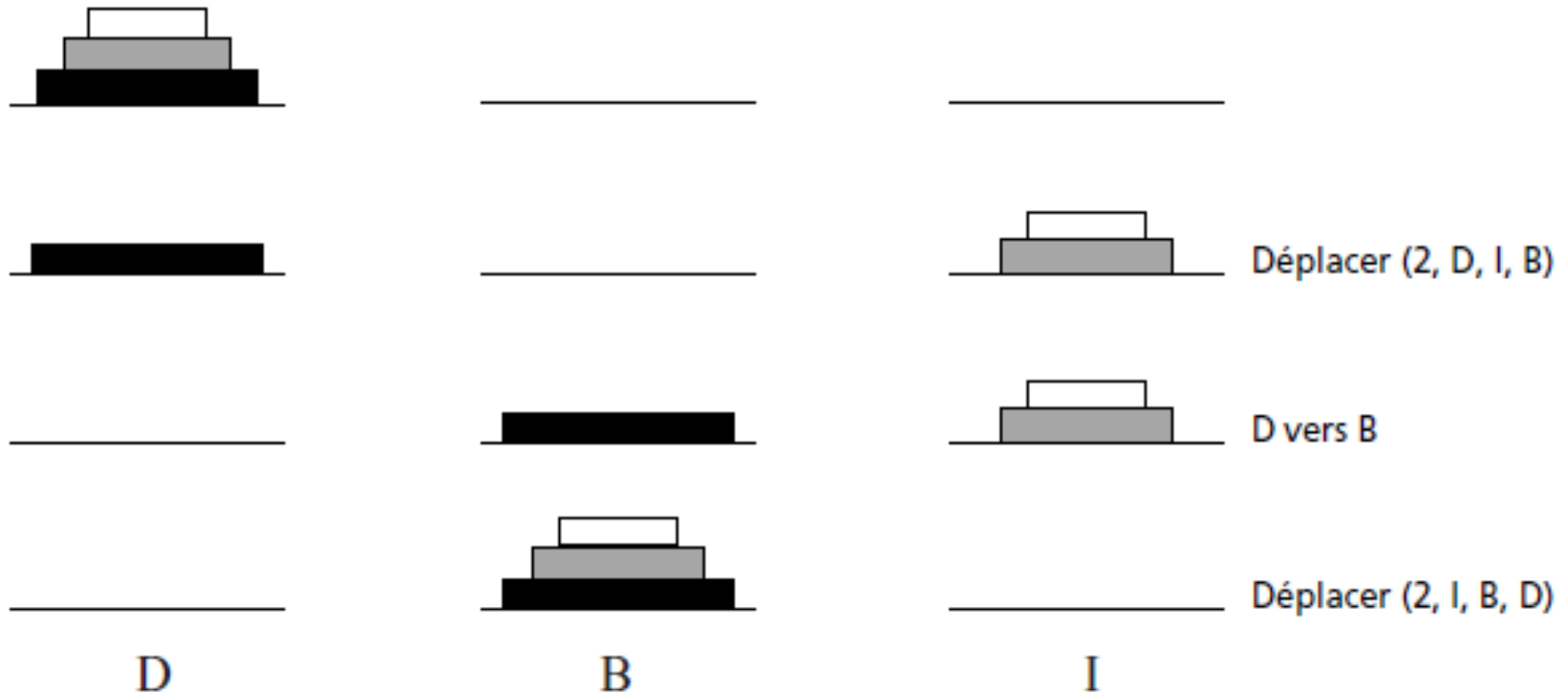
`deplacer(2, D, B, I);`



8. Fonctions récursives

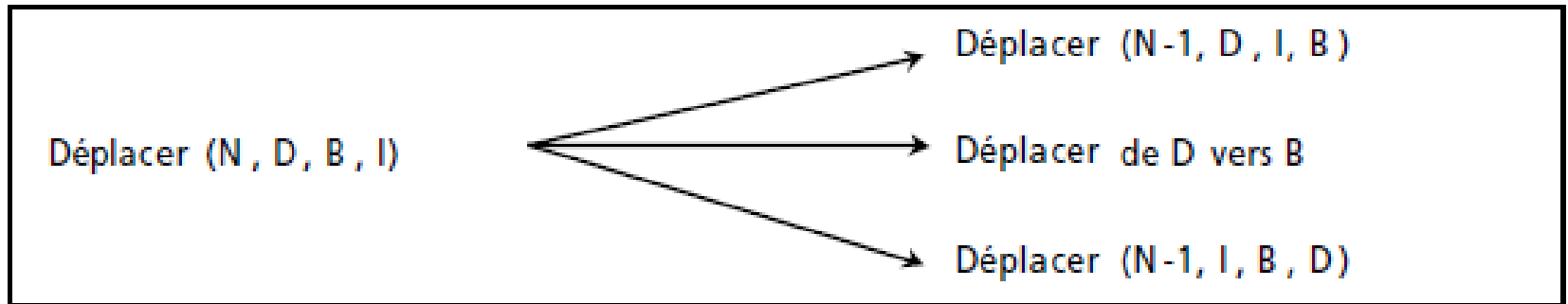
- Le cas de 3 disques: Pour déplacer 3 disques, il faut déplacer les 2 disques en sommet de **D** vers **I** (en utilisant **B** comme intermédiaire), ensuite déplacer le disque reposant sur le socle de **D** vers **B**, et ramener les 2 disques mis de côté sur **I**, en sommet de **B**.

deplacer(3, D, B, I);



8. Fonctions récursives

- ☉ D'une manière générale, pour déplacer **N** disques de **D** vers **B**, il faut déplacer les **N-1** disques de **D** vers **I** en utilisant **B** comme intermédiaire, déplacer le disque restant de **D** vers **B**, ramener les **N-1** disques de **I** vers **B** en utilisant **D** comme intermédiaire.



```
void deplacer (int N, char D, char B, char I) {  
    if (n > 0) {  
        deplacer (N-1, D, I, B);  
        printf ("%c --> %c\n", D, B);  
        deplacer (N-1, I, B, D);  
    }  
}
```

An abstract graphic design featuring a dark background. A horizontal band with a light-to-dark gray gradient spans the width of the image. Above this band, several thick, flowing lines in magenta, cyan, and yellow sweep across the frame from left to right. The word "demo" is written in a white, italicized, sans-serif font on the left side of the gray band.

demo